Linde

Table des matières

Introduction : 3
Spécifications du projet: 4
Répartition des tâches : 6
Étude des Capteurs de Pression Atmosphérique et d'Anémomètres : 7
Capteur de pression atmosphérique: 8
Comparaison des Capteurs : BMP180 vs BMP280 : 9
Étude du capteur BMP280: 10
Étude du BMP280 : 11
Capteur anémomètre: 12
Conclusion : 15

Introduction :

Je vais vous présenter mon rapport de projet individuel. Je m’appelle Adam Boumaad, étudiant en deuxième année de BTS CIEL B au Campus Ozanam. Durant cette année, j’ai participé à un projet collectif portant sur l’étude et l’amélioration d’une station météo.

Le projet s’inscrit dans le cadre de l’installation d’un système d’alerte sur un vignoble. L’objectif est de reproduire et optimiser une solution existante en intégrant de nouvelles fonctionnalités, notamment en diversifiant les moyens de transmission des données météorologiques et des alertes.

Le principe consiste à limiter le risque de grêle en favorisant la formation de pluie. Dans un nuage susceptible de générer de la grêle, les noyaux de condensation de grande taille accélèrent la formation des gouttes de pluie, qui, en montant en altitude, se transforment en grêlons. Ces noyaux de condensation, souvent faits de petites impuretés, aident l'humidité dans l'air à se transformer en gouttes plus grosses. À cause de leur poids, ces gouttes se transforment en grêle lorsqu'elles montent dans les zones plus froides du nuage et gèlent.

Pour limiter ce phénomène, on utilise des sels hygroscopiques, qui attirent l'humidité de l'air et favorisent la formation de gouttes d'eau plus petites. Cela empêche la création de gros noyaux de condensation, réduisant ainsi la formation de grêle. En effet, en diminuant la taille des noyaux, les sels hygroscopiques évitent l'accumulation rapide d'humidité et la croissance des gouttes, ce qui permet de prévenir la formation de grêlons.

Lors d'une alerte, des ballons biodégradables gonflés à l’hélium, équipés de torches libérant ces sels, sont envoyés dans les zones orageuses à risque. Bien que cette méthode soit efficace, elle est coûteuse et n’est utilisée que lorsque le risque de grêle est élevé. La décision d'agir est généralement prise par le chef de culture, en concertation avec son équipe, après avoir analysé les données météorologiques.

À l’aide d’une plateforme numérique dotée d'un outil d'évaluation des risques de grêle permet d'identifier les nuages menaçant. Les viticulteurs, informés par SMS, peuvent suivre en temps réel le déplacement des cellules orageuses, avec trois niveaux d'alerte : faible, modéré et fort, pour signaler l'intensité du danger.

Spécifications du projet:

Dans cette partie, je vais expliquer les spécificités du projet, en le divisant en trois sections principales. Tout d'abord, la section **"Ce que le client veut"** détaillera les besoins du client, notamment la collecte de données météorologiques fiables, l'envoi d'alertes SMS avec différents niveaux de risque, ainsi que l'accès à un site web pour consulter ces données en temps réel. Ensuite, la section **"Aspects techniques"** abordera les solutions technologiques mises en place pour répondre à ces besoins, telles que les outils de collecte des données (station météo, GPS), le traitement des informations et la gestion des alertes. Enfin, la section **"Station-Vigne et Serveur-Manoir"** expliquera le rôle spécifique de la Station-Vigne, qui collecte les données météorologiques sur le terrain, et du Serveur-Manoir, qui centralise ces informations, permettant ainsi de gérer et de déclencher à distance les actions nécessaires, comme le lâcher de ballons anti-grêle. Ces trois parties permettent de comprendre les exigences du projet et la manière dont elles sont mises en œuvre techniquement.

En effet, je vais décrire les besoins et attentes du client. Le client désire avoir accès à des données météorologiques fiables provenant de chaque station-vigne installée sur le domaine. Il souhaite également recevoir des alertes par SMS concernant le risque de grêle, ces alertes étant classées en trois niveaux : faible, modéré et élevé. Le client doit pouvoir consulter l'ensemble des données collectées via un site web hébergé sur le serveur du domaine, appelé Serveur-Manoir. Il souhaite également avoir la possibilité de gérer ce serveur et d'accéder à la base de données qui enregistre quotidiennement les données météorologiques à des fins d'archivage. Enfin, le client veut pouvoir déclencher à distance le lâcher de ballons anti-grêle en cas de nécessité.

De plus, pour les aspects techniques le système a pour objectif de collecter les données météorologiques nécessaires à l'évaluation du risque de grêle, telles que la température, l'humidité relative, la pression atmosphérique et la vitesse du vent, depuis la Station-Vigne. Il doit également recueillir les informations de géolocalisation des données via GNSS et horodater ces informations. La Station-Vigne analyse ces données et évalue le risque de grêle. Le niveau de ce risque est ensuite transmis au Serveur-Manoir, situé sur le site de production. Une alerte SMS est envoyée aux responsables identifiés, tels que le chef de culture et le responsable informatique. Le système permet également une connexion via smartphone au site web du vignoble pour consulter les données météorologiques et toutes les informations pertinentes. Le risque de grêle peut être validé en croisant les données collectées avec celles provenant d'une base de données externe, comme Météo-France (optionnel). Le lâcher du ballon anti-grêle peut être déclenché à distance, soit

depuis un smartphone, soit depuis le poste central du vignoble. Enfin, les données météorologiques collectées sont transférées quotidiennement au serveur pour l’archivage et les statistiques. Le système doit également intégrer une gestion de l’autonomie énergétique de la Station-Vigne, en utilisant des solutions comme des panneaux solaires et des batteries, afin de garantir un fonctionnement continu même en l'absence de connexion au réseau électrique.

Enfin, pour les deux entités secondaires qui sont la station-vigne et le serveur manoir. Au niveau de la Station-Vigne, il est essentiel de mesurer les principales données météorologiques telles que la température, l'humidité, la pression atmosphérique et la vitesse du vent. Une plateforme de traitement des données, à faible consommation d'énergie comme Arduino ou une alternative similaire, doit être mise en place. Des nœuds LoRa doivent être installés et connectés à une passerelle LoRa située au Serveur-Manoir. Un module GPS doit également être intégré pour fournir des données de localisation, et un service d’envoi d’alertes par SMS doit être activé. En outre, la station doit être équipée d’un système garantissant son autonomie énergétique, par exemple avec des panneaux solaires ou des batteries.

Du côté du Serveur-Manoir, il est nécessaire de déployer une infrastructure réseau complète, incluant un serveur principal, une passerelle LoRa et un PC dédié à l’administration. Le serveur sera chargé de collecter, analyser et archiver les informations transmises par la Station-Vigne. Enfin, un site web devra être conçu pour permettre la visualisation des données recueillies, qu'elles soient en temps réel, quotidiennes ou sauvegardées.

Répartition des tâches :

Étudiant 1 : Benoît Borek

Phase 1 : Étude capteur de température et Humidité.

Phase 2 : GSM,Lora,lier GNSS et anémomètre.

Phase 3 : Inscription serveur TTN,passerelle Lora,afficher données(tableau,graphique).

Étudiant 2 : Boumaad Adam

Phase 1 : Étude capteur de pression atmosphérique,anémomètre.

Phase 2 : Lier capteur de température,Humidité et pression atmosphérique.

Phase 3 : Site web(page d'accueil,page archivage,page déclenchement).

Étudiant 3 : Alban

Phase 1 : Étude GPS,Horodatation.

Phase 2 : Batterie,panneau solaire,Régulateur.

Phase 3 : 433 Mhz,Réalisation opération de maintenance et/ou de réparation tester la fonction déclenchement.

Étude des Capteurs de Pression Atmosphérique et d'Anémomètres :

Dans le cadre de la première phase du projet, en tant qu'étudiant numéro 2, ma tâche principale consiste à réaliser une étude approfondie sur les capteurs de pression atmosphérique et les anémomètres.

Cette phase vise à comprendre le fonctionnement de ces capteurs essentiels dans la mesure des variables environnementales. Je vais commencer par analyser les différents types de capteurs de pression atmosphérique, en examinant leurs caractéristiques techniques telles que leur précision, leur plage de mesure et leurs applications dans des domaines comme la météorologie et les systèmes de contrôle de l'environnement.

De même, l'étude des anémomètres, utilisés pour mesurer la vitesse du vent, permettra de comprendre leur fonctionnement, leurs avantages, et leur fiabilité dans des conditions variables.

À travers cette étude, mon objectif est de sélectionner les capteurs les plus adaptés pour les étapes suivantes du projet, en mettant l'accent sur la fiabilité et la précision des données qu'ils génèrent.

Capteur de pression atmosphérique:

Tout d'abord, l'utilisation du capteur de pression atmosphérique est primordiale pour la détection d'une tombée de grêle. La pression atmosphérique est un indicateur clé des conditions météorologiques, car une baisse rapide de la pression peut signaler l'approche d'une perturbation, comme un orage ou une cellule grêleuse. Les variations de pression sont directement liées aux mouvements de l'air et à la formation de nuages. Des conditions de basse pression favorisent l'ascension de l'air chaud, ce qui peut créer des nuages cumulonimbus, de grands nuages orageux responsables de la grêle. Ainsi, la pression atmosphérique joue un rôle crucial dans l'analyse de la formation des nuages et des risques d'orage.

Combiné avec d'autres capteurs, tels que ceux mesurant la température, l'humidité relative et la vitesse du vent, un capteur de pression permet d'améliorer l'évaluation des risques de grêle. Les conditions idéales pour la formation de grêle peuvent être mieux modélisées grâce à ces données précises de pression.

En outre, des seuils de pression spécifiques, associés à d'autres facteurs comme la vitesse du vent, peuvent être utilisés pour déclencher des alertes SMS afin d'avertir les viticulteurs en temps réel. Enfin, pour le capteur de pression atmosphérique choisi est un capteur de pression moderne, comme le BMP280, sont peu coûteux, à faible consommation d'énergie et offrent une grande précision. Ces caractéristiques les rendent idéaux pour une station météorologique autonome, apportant ainsi à la fois efficacité et fiabilité dans la gestion des risques liés à la grêle.

Cependant, avant d'avoir choisi le capteur BMP280 j'ai également examiné le capteur de pression atmosphérique BMP180, un modèle plus ancien de la même gamme de capteurs . J'ai étudié ses spécifications techniques et les ai comparées à celles du BMP280 à l'aide d'un tableau, un capteur plus récent et plus performant. Enfin j'ai trouvé un document technique qui explique les fonctionnalités de ce capteur.

Lien de la datasheet : https://cdn-shop.adafruit.com/datasheets/BST-BMP280-DS001-11.pdf

Comparaison des Capteurs : BMP180 vs BMP280 :

Paramètres	BMP180	BMP280	Avantage par rapport au projet
Dimensions	3.6 × 3.8 mm	2.0 × 2.5 mm	Le BMP280 est plus compact, facilitant l'intégration.
Consommation de courant	12 µA	2.7 µA	Le BMP280 est 4 fois plus économe en énergie, idéal pour une station autonome.
Résolution de pression	1 Pa	0,16 Pa	Le BMP280 permet des mesures de pression plus précises, cruciales pour évaluer le risque de grêle.
Options de filtrage	Aucune	Cinq bandes passantes	Le BMP280 réduit le bruit dans les mesures, augmentant la fiabilité des données.

Finalement, le BMP180 offre des caractéristiques intéressantes, mais sa résolution de pression est inférieure à celle du BMP280, avec une précision de 1 Pa contre 0,16 Pa pour le BMP280. De plus, le BMP280 offre une plage de fonctionnement plus large, une meilleure résolution de température et des options de filtrage plus avancées.
En comparant les deux capteurs, j'ai noté que le BMP280 consomme moins d'énergie, avec une consommation de 2,7 µA contre 12 µA pour le BMP180. Il dispose également de meilleures interfaces, permettant une communication via I^2C et SPI, et offre une plus grande flexibilité pour les projets nécessitant de multiples modes de communication.

Après avoir pris en compte ces critères, j'ai finalement opté pour le BMP280 pour mon projet de station météorologique, car il offre une meilleure précision, une consommation d'énergie plus faible et une plus grande compatibilité avec les besoins spécifiques du projet.

Étude du capteur BMP280:

Le BMP280 est un capteur de pression atmosphérique et de température, alimenté par une tension de 3,3 V. Il dispose de caractéristiques telles que le suréchantillonnage et prend en charge les interfaces I2C et SPI, offrant ainsi une communication flexible avec différents microcontrôleurs.

Étude matérielle :

Le module BMP280 est conçu de la manière suivante : au centre se trouve le capteur, qui possède un petit trou permettant de mesurer la pression atmosphérique. Autour de ce capteur, on trouve 4 résistances et 2 condensateurs. Le module comporte 6 broches : VCC, GND, SCL, SDA, CSB et SDO.

Pins	Description
VIN	Il sert d'alimentation d'entrée. (+3,3V).
GND	La masse du module.
SCL	Entrée pour l'horloge.
SDA	Entrée/Sortie pour les données.
CSB	Chip Select (SPI)
SDO	Data out

Étude du BMP280 :

Maintenant je dois comprendre le fonctionnement du capteur lui-même. En utilisant la documentation technique que j’avais trouvé auparavant.
Lien : https://cdn-shop.adafruit.com/datasheets/BST-BMP280-DS001-11.pdf

1) Le BMP280 est un capteur de pression atmosphérique et température qui intègre un convertisseur analogique-numérique (CAN). Ce composant est essentiel car il permet de convertir les signaux analogiques générés par le capteur de pression en données numériques, facilitant ainsi leur traitement par un microcontrôleur. Cette conversion est cruciale pour les applications de mesure, car elle permet d'obtenir des valeurs précises et exploitables pour l'analyse des conditions météorologiques.

2) Le BMP280 propose trois modes de fonctionnement principaux, chacun adapté à des besoins spécifiques. Le **Mode Sleep** est celui par défaut, où le capteur ne réalise aucune mesure, permettant ainsi de minimiser la consommation d’énergie lorsque des mesures fréquentes ne sont pas nécessaires.
- Mode Sleep : Il est particulièrement utile pour prolonger la durée de vie de la batterie.
Ensuite, en **Mode Normal**, le capteur alterne entre des périodes actives de mesure et des périodes de veille, selon les paramètres configurés.
- Mode Normal : Ce mode est adapté aux applications qui nécessitent des mesures régulières et continue, comme dans le cas de la surveillance de conditions météorologiques. Enfin, le **Mode Forced** permet d'effectuer une seule mesure, après quoi le capteur retourne automatiquement en mode Sleep.
- Mode Forced : Ce mode est idéal pour des applications où des mesures ponctuelles sont nécessaires, offrant ainsi une solution sur demande pour des situations spécifiques.

3) Le BMP280 intègre un filtre numérique IIR (Infinite Impulse Response) conçu pour réduire les fluctuations à court terme des données de pression. Le coefficient de filtrage de ce filtre peut être configuré afin de s'adapter aux besoins spécifiques de l'application. Ce filtrage est crucial pour obtenir des données météorologiques précises, car il aide à minimiser l'impact des perturbations environnementales, telles que les effets du vent ou les mouvements soudains. De plus, le filtre peut être ajusté en fonction des conditions particulières, offrant ainsi une solution flexible pour des environnements instables ou variables.

4) Dans cette partie, je vais me focaliser sur la documentation liée à l'interface I2C du BMP280, car j'ai choisi cette le bus I2C pour le câblage et la programmation. Je vais examiner en détail les informations fournies dans la datasheet concernant l'utilisation de l'I2C, telles que les adresses de communication, la configuration des registres et les étapes nécessaires pour établir une connexion stable entre le capteur et le microcontrôleur. Cela inclut également la gestion de la lecture et de l'écriture des données sur les lignes **SCL** et **SDA** afin d'assurer une transmission correcte des informations de pression et de température. L'objectif est d'appliquer précisément les instructions de la datasheet pour optimiser le fonctionnement du capteur en I2C.
Le choix de l'interface I²C pour la connexion du BMP280 présente plusieurs avantages importants. En termes de **simplicité de connexion**, l'I²C nécessite seulement deux fils pour la communication (SDA et SCL), plus un pour la terre (GND), contrairement à l'interface SPI qui demande au moins quatre fils (MOSI, MISO, SCK, CSB), ce qui complique le câblage, notamment lorsqu'il s'agit de connecter plusieurs capteurs. En ce qui concerne **l'économie d'énergie**, Le BMP280 consomme **2,7 µA** (microampères) en mode **I²C** lorsqu'il est configuré pour échantillonner les données à **1 Hz** en mode **Normal**. Cela signifie que le capteur prend une mesure de pression et de température une fois par seconde (1 échantillon par seconde) et que sa consommation d'énergie est très faible pendant cette période. Bien que la consommation en mode SPI ne soit pas précisée directement, les capteurs utilisant SPI consomment généralement plus d'énergie en raison de la nécessité de maintenir plusieurs lignes actives.
L'interface I²C permet également une **facilité de mise en réseau**, en permettant de connecter jusqu'à 127 dispositifs sur le même bus, avec une adresse unique pour chaque appareil. Le BMP280 peut être configuré avec deux adresses I²C différentes (0x76 et 0x77) selon l'état de la broche SDO. En revanche, en SPI, chaque dispositif nécessite une ligne CS (Chip Select) distincte, limitant ainsi le nombre de capteurs pouvant être connectés simultanément et compliquant le câblage. En termes de **vitesse de communication**, l'I²C supporte des vitesses allant jusqu'à 3,4 MHz (mode haute vitesse), ce qui est largement suffisant pour les besoins de mesure du BMP280, même si le SPI peut atteindre des vitesses plus élevées (jusqu'à 10 MHz selon la datasheet).

Du côté de la **complexité de l'interface**, l'I²C présente un protocole plus simple avec des transactions de lecture et d'écriture gérées par le maître, réduisant ainsi la complexité du code et le risque d'erreurs. À l'inverse, l'interface SPI requiert une gestion plus complexe des lignes de données et de l'horloge, ce qui peut augmenter le temps de développement. En ce qui

concerne le **support de filtrage numérique**, le BMP280 utilise un filtre IIR intégré pour minimiser les fluctuations à court terme dans les données de pression. Ce filtrage est plus facilement géré dans un environnement I²C, où les données peuvent être lues et traitées en continu sans nécessiter de multiples transactions SPI.

En conclusion, l'I²C est l'option la plus adaptée pour mon projet de station météo, en raison de sa simplicité de connexion, de son efficacité énergétique, de sa capacité à gérer plusieurs capteurs sur un même bus, et de sa vitesse de communication suffisante pour les applications de mesure. Ces avantages font de l'I²C un choix judicieux, notamment dans un contexte où l'économie d'énergie et la facilité d'intégration sont essentielles.

Dans cette partie, je vais me concentrer sur les registres du BMP280 qui seront importants pour mon code. Ces registres permettent de configurer le capteur, récupérer les données et ajuster des paramètres comme le mode de fonctionnement, le taux d'échantillonnage et le filtrage numérique.

Tout d'abord,Les adresses I²C **0x76** et **0x77** du BMP280 sont utilisées respectivement lorsque la broche SDO est connectée à la terre (GND) et à VDDIO (tension d'alimentation), permettant ainsi la communication avec le capteur sur le bus I²C. Ces adresses uniques sont essentielles pour éviter les conflits lors de la connexion de plusieurs capteurs, garantissant un fonctionnement correct dans un système de mesure.

Register 0xF4 *"ctrl_meas"*	Name	Description
Bit 7, 6, 5	osrs_t[2:0]	Controls oversampling of temperature data. See chapter 3.3.2 for details.
Bit 4, 3, 2	osrs_p[2:0]	Controls oversampling of pressure data. See chapter 3.3.1 for details.
Bit 1, 0	mode[1:0]	Controls the power mode of the device. See chapter 3.6 for details.

Ensuite **Registre de Contrôle (0xF4) :** Ce registre configure les options d'acquisition de données, y compris les paramètres d'oversampling pour la pression et la température, ainsi que le mode de fonctionnement (Sleep, Normal, Forced). Cela permet une configuration facile via I²C.

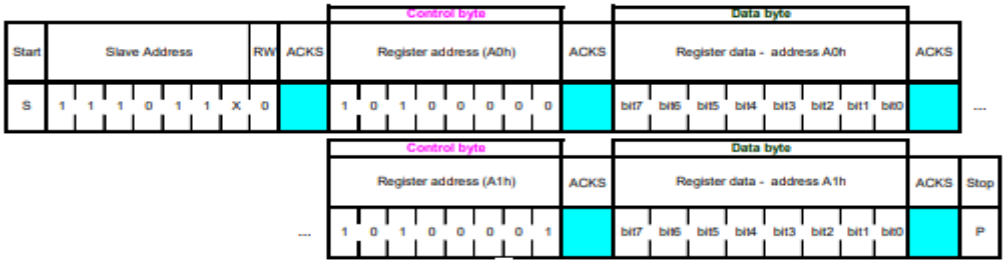
Register 0xF7-0xF9 *"press"*	Name	Description
0xF7	press_msb[7:0]	Contains the MSB part up[19:12] of the raw pressure measurement output data.
0xF8	press_lsb[7:0]	Contains the LSB part up[11:4] of the raw pressure measurement output data.
0xF9 (bit 7, 6, 5, 4)	press_xlsb[3:0]	Contains the XLSB part up[3:0] of the raw pressure measurement output data. Contents depend on temperature resolution, see table 5.

Enfin Les registres **0xF7**, **0xF8**, et **0xF9** du BMP280 contiennent respectivement les 8 bits les plus significatifs (MSB), les 8 bits suivants (LSB), et les 4 bits les moins significatifs (XLSB) de la mesure de pression atmosphérique.

5) Trame de Communication I²C avec le BMP280.

Dans une communication I²C, le **maître (Master)** est le dispositif qui initie et contrôle le flux de données, comme un microcontrôleur (par exemple, un Arduino ou Raspberry Pi) qui envoie des commandes aux capteurs. Le **esclave (Slave)**, comme le BMP280 dans ce cas, répond aux requêtes du maître en envoyant des données de pression et de température en retour, permettant ainsi une interaction fluide entre les dispositifs sur le bus I²C.

-Écriture dans le Registre I²C :

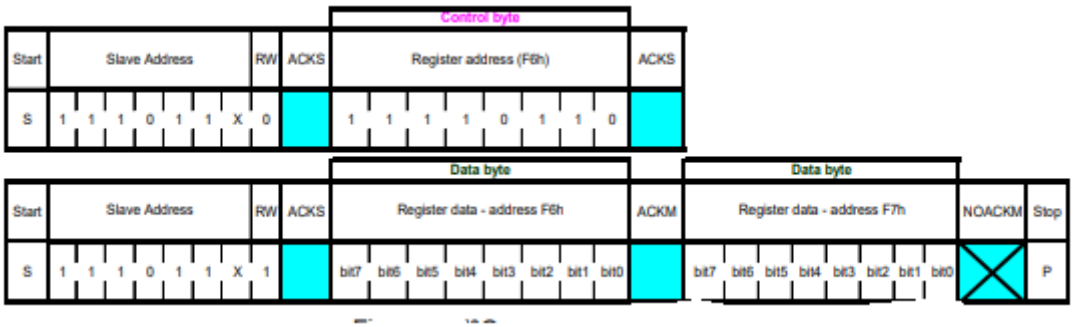


Le maître commence la communication en générant un signal de démarrage(bit start), suivi de l'envoi de l'adresse de l'esclave en mode écriture (RW = '0'). L'esclave répond par un accusé de réception (ACK). Envoi des Données
Le maître envoie ensuite les paires d'adresses de registre et de données, chaque envoi étant confirmé par un ACK de l'esclave. La transaction se termine par un signal d'arrêt(bit de stop), indiquant la fin de l'envoi des données.

Exemple d'Écriture I²C (Configuration du Registre F4 pour la Pression) :

1. Démarrage de la Communication (Start Condition) :
    - Le maître (par exemple, un Arduino) génère un signal de démarrage (bit Start) pour initier la communication sur le bus I²C.
2. Envoi de l'Adresse de l'Esclave :
    - Le maître envoie l'adresse de l'esclave (par exemple, 0x76 ou 0x77 pour le BMP280), en mode écriture (RW = '0'). Cela indique au BMP280 que le maître veut lui envoyer des données.
    - L'esclave (le BMP280) répond par un accusé de réception (ACK) pour signaler qu'il est prêt à recevoir des données.
3. Envoi des Données :
    - Le maître envoie ensuite des paires d'adresses de registre et de données. Par exemple, pour configurer le BMP280, le maître pourrait vouloir écrire dans le registre F4 pour configurer la fréquence d'échantillonnage.
    - Adresse de registre F4 : Le maître envoie 0xF4 (adresse du registre de contrôle du BMP280).
    - Valeur à écrire dans le registre : Le maître envoie ensuite une valeur, par exemple 0x27, pour définir certains paramètres (comme la fréquence d'échantillonnage).
    - Après chaque envoi de donnée, l'esclave répond avec un ACK pour confirmer qu'il a bien reçu l'adresse et la donnée.
4. Fin de la Transaction (Stop Condition) :
    - Une fois toutes les données envoyées, le maître termine la transaction par un signal d'arrêt (bit Stop). Cela indique que l'envoi des données est terminé.

-Lecture dans le Registre I²C :



Le maître envoie d'abord l'adresse I²C de l'esclave en mode écriture (RW = '0') et spécifie l'adresse du registre à lire. Ensuite, il génère un signal de démarrage ou une condition de reprise pour préparer la lecture.

Passage en Mode Lecture. Le maître adresse ensuite l'esclave en mode lecture (RW = '1'), et l'esclave répond par un accusé de réception (ACK). L'esclave envoie les données demandées, et le maître envoie un ACK après chaque octet reçu. À la fin, le maître envoie un NACK pour signaler qu'il a terminé la lecture.

<u>Exemple de Lecture I²C du BMP280 (Registre de la Pression) :</u>

1. Envoi de l'Adresse de Registre :

    - Le maître (par exemple, un Arduino) envoie l'adresse I²C du BMP280 en mode écriture (RW = '0').
    - Ensuite, le maître spécifie l'adresse du registre qu'il veut lire. Par exemple, pour lire les données de pression, le registre de pression est à 0xF7.

Séquence :

- Start Condition → Le maître démarre la communication.
- Adresse du capteur (0x76 ou 0x77) → Le maître envoie l'adresse de l'esclave.
- Adresse du registre (0xF7) → Le maître envoie l'adresse du registre de pression à lire.
- ACK → L'esclave répond par un accusé de réception.

2. Passage en Mode Lecture :

    - Une fois l'adresse du registre envoyée, le maître génère un signal de reprise (Repeated Start Condition) ou d'arrêt pour initier la lecture.
    - Le maître adresse ensuite l'esclave en mode lecture (RW = '1') pour demander la donnée du registre spécifié.

Séquence :

- Repeated Start → Le maître envoie un signal de reprise pour continuer la lecture.
- Adresse du capteur (0x76 ou 0x77) → Le maître adresse à nouveau l'esclave, mais cette fois en mode lecture.
- ACK → L'esclave répond avec un accusé de réception.

3. Envoi des Données :

- L'esclave (BMP280) envoie les octets de données demandées. Par exemple, il peut renvoyer 3 octets pour les données de pression (et de température, selon la configuration).
- Le maître reçoit chaque octet et envoie un ACK après chaque octet pour indiquer qu'il a bien reçu la donnée.

4. Fin de la Lecture :

- Une fois que tous les octets ont été envoyés, le maître envoie un NACK après le dernier octet pour indiquer qu'il a terminé la lecture.
- Enfin, le maître envoie un signal d'arrêt pour terminer la communication.

Câblage:

Dans un premier temps, je vais câbler mon capteur BMP280 à mon Raspberry Pi via l'interface I2C. Le BMP280 utilise deux fils pour la communication : SDA (Serial Data) pour les données et SCL (Serial Clock) pour l'horloge. En plus de ces deux fils, je vais connecter les pins GND (masse) et VCC (alimentation) du capteur. Puisque le BMP280 fonctionne à 3,3V, je vais l'alimenter avec la broche 3,3V du Raspberry Pi.Cela me permet de relier le capteur à un microcontrôleur plus puissant et plus adapté aux besoins de mon projet, conformément aux exigences de mon professeur.

Dans cette deuxième partie, je vais expliquer le programme en détail, en décrivant chaque section du code et son fonctionnement.

1)Le code commence par des directives d'inclusion pour importer les bibliothèques nécessaires. `#include <iostream>` permet d'utiliser `std::cout` et `std::cerr` pour l'affichage. `#include <fcntl.h>` et `#include <unistd.h>` sont essentiels pour les opérations sur les

fichiers et les appels système POSIX. #include <sys/ioctl.h> permet de contrôler des périphériques, et #include <linux/i2c-dev.h> fournit des définitions pour les opérations I²C.*

Le code définit ensuite des constantes : #define BMP280_ADDRESS 0x76 pour l'adresse I²C du BMP280, #define REG_CONTROL 0xF4 pour le registre de contrôle, et #define REG_PRESS_MSB 0xF7 pour le registre de pression.

```
#include <iostream>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <linux/i2c-dev.h>

#define BMP280_ADDRESS 0x76
#define REG_CONTROL 0xF4
#define REG_PRESS_MSB 0xF7
```

2)Dans cette section du code, nous initialisons la communication avec le capteur BMP280 via le bus I²C. La ligne const char *device = "/dev/i2c-1"; définit le chemin du périphérique I²C, tandis que int file = open(device, O_RDWR); ouvre le bus en mode lecture/écriture. Si l'ouverture échoue, un message d'erreur est affiché et le programme se termine avec un code d'erreur **1**. Ensuite, ioctl(file, I2C_SLAVE, BMP280_ADDRESS) configure le descripteur de fichier pour communiquer avec le capteur en définissant son adresse I²C. En cas d'échec de cette opération, un message d'erreur est également affiché, et le programme se termine. Cette partie du code garantit ainsi une communication fiable avec le capteur BMP280.

```
int main() {
    const char *device = "/dev/i2c-1";
    int file = open(device, O_RDWR);
    if (file < 0) {
        std::cerr << "Échec de l'ouverture du bus I2C\n";
        return 1;
    }

    if (ioctl(file, I2C_SLAVE, BMP280_ADDRESS) < 0) {
        std::cerr << "Échec de l'accès au bus ou de la communication avec l'esclave\n";
        return 1;
    }
```

3)Dans cette section, nous configurons le capteur BMP280. La ligne char config[2] = {REG_CONTROL, 0x27}; définit les paramètres d'oversampling et

le mode de fonctionnement. Le premier élément est l'adresse du registre de contrôle, et le second, 0x27, configure le capteur en mode normal avec un oversampling de x1. La fonction `write(file, config, 2)` envoie ces paramètres au capteur via le bus I²C. Si l'écriture échoue, un message d'erreur est affiché et le programme se termine avec un code d'erreur **1**. Cette partie garantit que le BMP280 est correctement configuré pour effectuer des mesures.

```
// Configurer le BMP280 : définir l'oversampling et le mode
char config[2] = {REG_CONTROL, 0x27};  // Mode normal, oversampling x1
if (write(file, config, 2) != 2) {
    std::cerr << "Échec de la configuration du BMP280\n";
    return 1;
```

Méthode pour configurer Mon capteur à l'aide de la datasheet :
Pour cela j'ai vu sur la dataseet qui permet de configurer le capteur c'est le registre F4.

Register Name	Address	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	Reset state
temp_xlsb	0xFC	temp_xlsb<7:4>				0	0	0	0	0x00
temp_lsb	0xFB	temp_lsb<7:0>								0x00
temp_msb	0xFA	temp_msb<7:0>								0x80
press_xlsb	0xF9	press_xlsb<7:4>				0	0	0	0	0x00
press_lsb	0xF8	press_lsb<7:0>								0x00
press_msb	0xF7	press_msb<7:0>								0x80
config	0xF5	t_sb[2:0]			filter[2:0]				spi3w_en[0]	0x00
ctrl_meas	0xF4	osrs_t[2:0]			osrs_p[2:0]			mode[1:0]		0x00
status	0xF3					measuring[0]			im_update[0]	0x00
reset	0xE0	reset[7:0]								0x00
id	0xD0	chip_id[7:0]								0x58
calib25...calib00	0xA1...0x88	calibration data								individual

Registers:	Reserved registers	Calibration data	Control registers	Data registers	Status registers	Revision	Reset
Type:	do not write	read only	read / write	read only	read only	read only	write only

Donc en voyant F4 : on peut apercevoir qu'il nous faut trois autres informations osrs_t osrs_p, mode

Table 22: Register settings osrs_t

osrs_t[2:0]	Temperature oversampling
000	Skipped (output set to 0x80000)
001	oversampling ×1
010	oversampling ×2
011	oversampling ×4
100	oversampling ×8
101, 110, 111	oversampling ×16

osrs_p[2:0]	Pressure oversampling
000	Skipped (output set to 0x80000)
001	oversampling ×1
010	oversampling ×2
011	oversampling ×4
100	oversampling ×8
101, Others	oversampling ×16

mode[1:0]	Mode
00	Sleep mode
01 and 10	Forced mode
11	Normal mode

Etant donné que j'ai choisi le oversmpling x1 et le mode normal en les mettant dans l'ordre en trouve :
Ob 0010 0111 -> 0x 27

4) Dans cette section, nous définissons une variable pour le nombre de relevés à effectuer, initialisée à 15 avec `int nombreReleves = 15;`, mais cette valeur peut être modifiée selon les besoins. Ensuite, une boucle `for` est utilisée pour itérer sur le nombre de relevés spécifié. La boucle commence avec `int i = 0;`, où `i` sert de compteur, et continue tant que `i` est inférieur à `nombreReleves`. À chaque itération, `i` est incrémenté de 1 grâce à `i++`. À l'intérieur de la boucle, la fonction `usleep(1000);` est appelée pour introduire une pause de 1000 microsecondes (ou 1 milliseconde), permettant ainsi d'attendre que la mesure soit terminée avant de procéder à la prochaine opération

```
// Variable pour le nombre de relevés
int nombreReleves = 15; // Changez cette valeur pour le nombre de relevés souhaité

for (int i = 0; i < nombreReleves; i++) {
    // Attendre que la mesure soit terminée
    usleep(1000);
```

5) Dans cette section du code, nous lisons les données de pression du capteur BMP280. La variable `char reg = REG_PRESS_MSB;` est définie pour indiquer le registre à lire. La fonction `write(file, &reg, 1)` envoie cette adresse de registre au capteur. Si l'écriture échoue, un message d'erreur est affiché et le programme se termine. Ensuite, un tableau `char data[6];` est créé pour stocker les données lues. La fonction `read(file, data, 6)` lit 6 octets de données à partir du capteur. Si la lecture échoue, un message d'erreur est également affiché. Les données de pression sont ensuite combinées à partir des octets lus pour calculer la pression brute avec l'expression `int rawPressure = ((int)data[0] << 12) | ((int)data[1] << 4) | ((data[2] >> 4) & 0x0F);`. Enfin, le résultat brut de la pression est affiché en pascals avec `std::cout`.

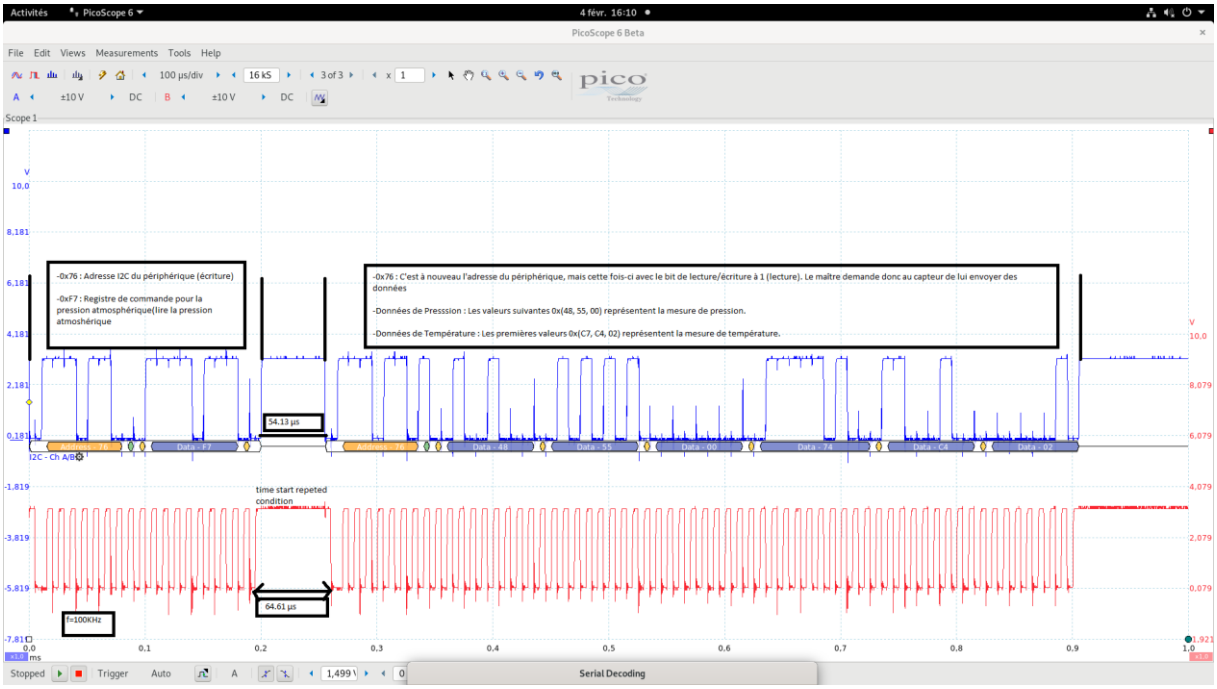
```
// Lire les données de pression
char reg = REG_PRESS_MSB;
if (write(file, &reg, 1) != 1) {
    std::cerr << "Échec de la définition du pointeur de registre\n";
    return 1;
}

char data[6];
if (read(file, data, 6) != 6) {
    std::cerr << "Échec de la lecture des données du BMP280\n";
    return 1;
}

// Combiner les données pour calculer la pression brute
int rawPressure = ((int)data[0] << 12) | ((int)data[1] << 4) | ((data[2] >> 4) & 0x0F);

// Afficher le résultat brut en Pa
std::cout << "Relevé " << (i + 1) << ": Pression brute = " << rawPressure << " Pa" << std::endl;
```

D'après la datasheet, les bits de poids forts se situent entre les positions [19 ; 12], les bits de poids faibles entre [11 ; 4], et les bits les moins significatifs entre [3 ; 0]. Les autres bits sont à ignorer.

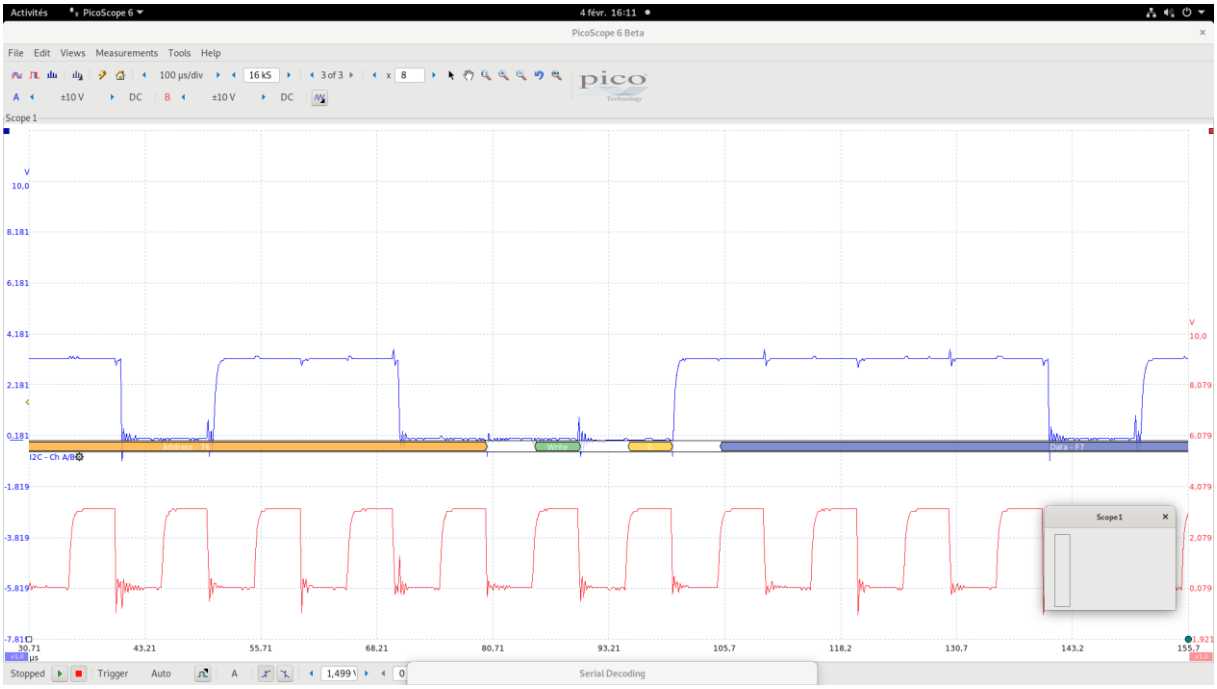
Register 0xF7-0xF9 *"press"*	**Name**	**Description**
0xF7	press_msb[7:0]	Contains the MSB part up[19:12] of the raw pressure measurement output data.
0xF8	press_lsb[7:0]	Contains the LSB part up[11:4] of the raw pressure measurement output data.
0xF9 (bit 7, 6, 5, 4)	press_xlsb[3:0]	Contains the XLSB part up[3:0] of the raw pressure measurement output data. Contents depend on temperature resolution, see table 5.

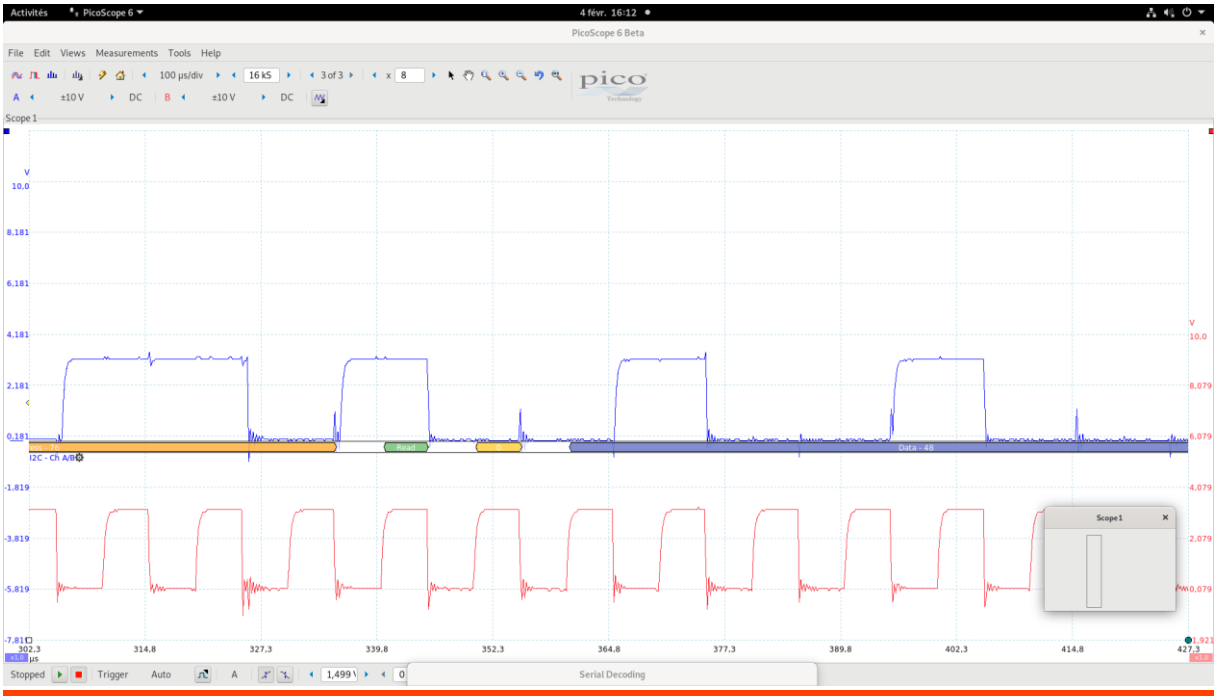
Picoscope :

Dans cette dernière partie, je vais relever les chronogrammes à l'aide d'un PicoScope 6 pour analyser les différentes phases, telles que la transition du maître, l'écriture et la lecture, ainsi que la condition de "Répète Start".



De plus, je vais présenter d'autres relevés pour illustrer le passage du maître du mode écriture au mode lecture.

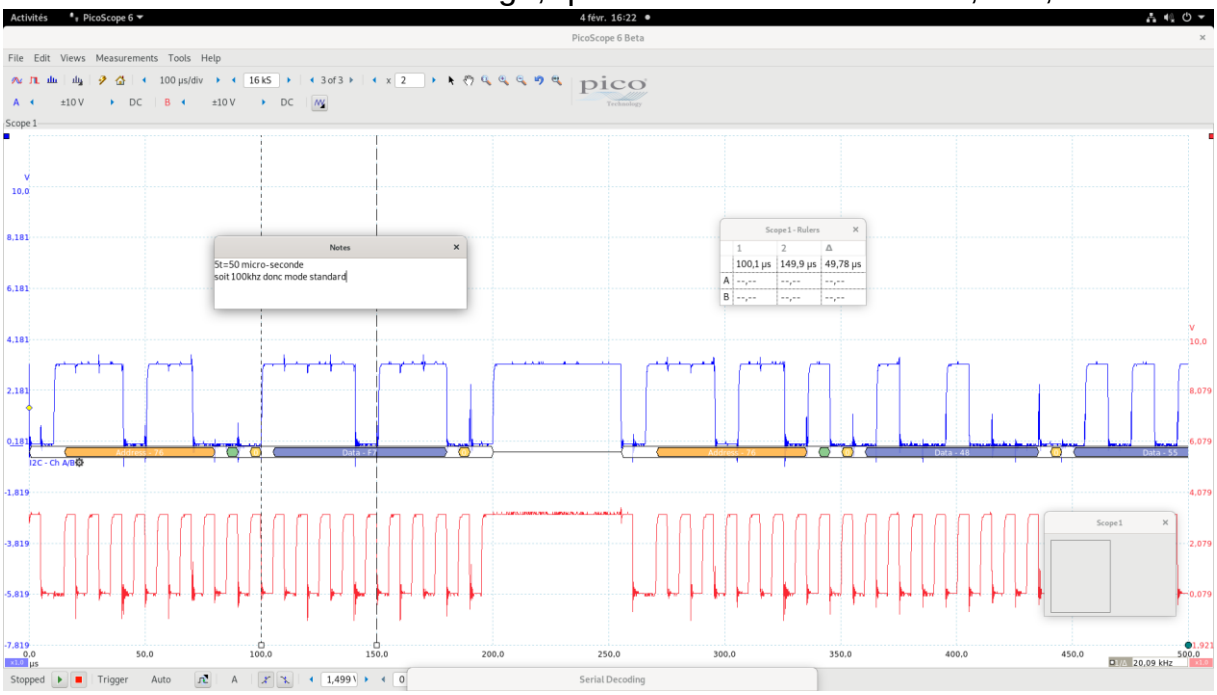




Vert : mode écriture(0)/lecture(1)

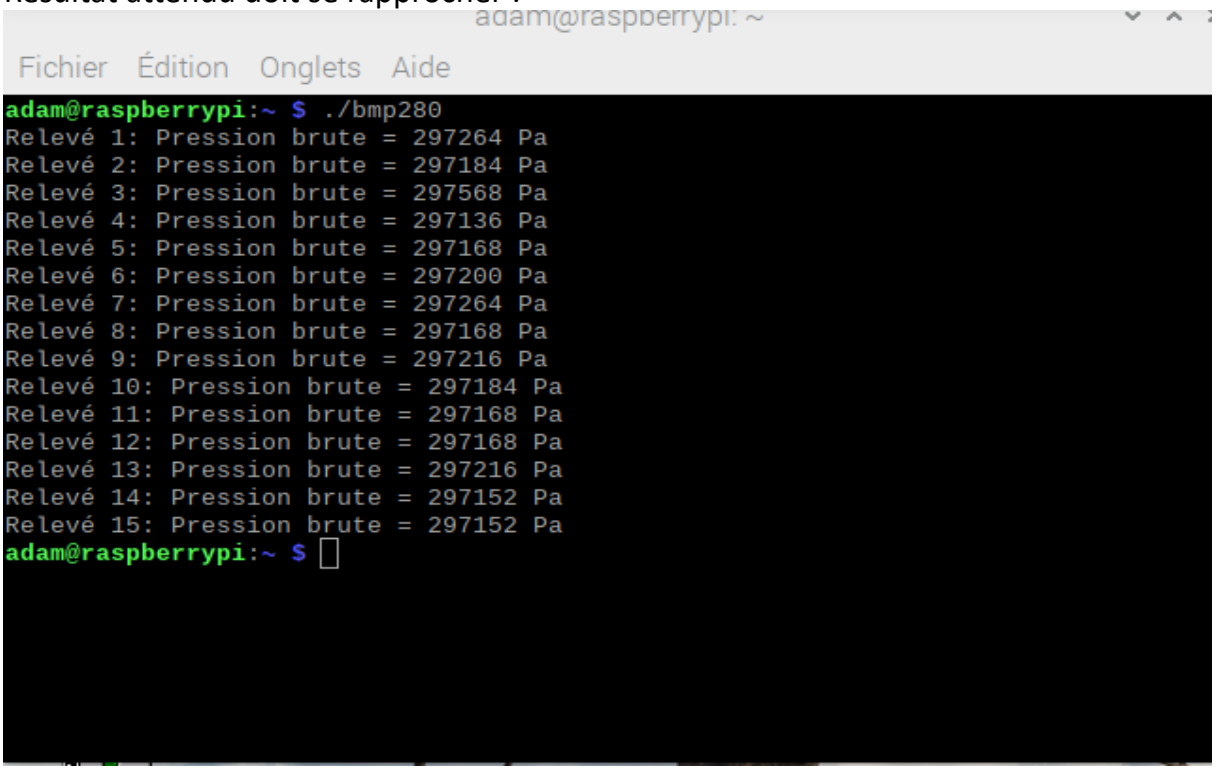
Jaune : ACK (0) Après chaque octet envoyé, le récepteur (esclave) doit renvoyer un bit ACK (0) pour signaler qu'il a correctement reçu l'octet. Si le récepteur n'est pas prêt ou ne reçoit pas les données correctement, il peut répondre avec un bit NACK (Not Acknowledged) (1), ce qui indique un échec de la réception.

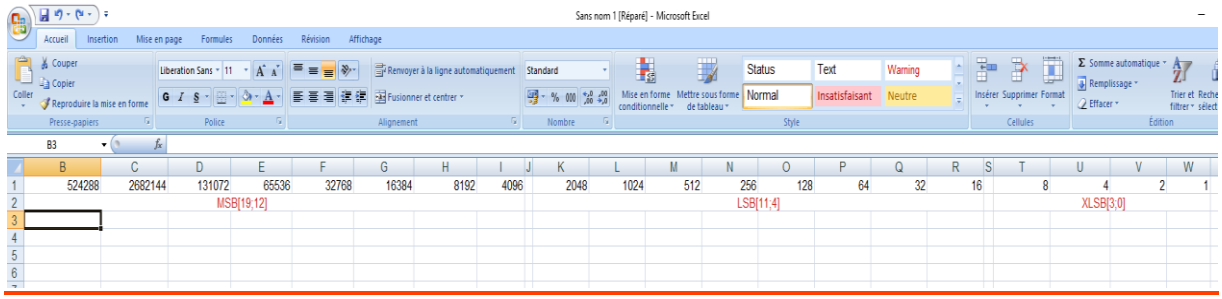
Enfin, j'ai calculé la période afin de déterminer la fréquence de l'horloge et ainsi identifier le mode de l'horloge, qu'il soit en mode standard, fast, etc.



Dans cette étape, je vais calculer la pression à la main à l'aide d'un fichier Excel et vérifier si les résultats correspondent à ceux affichés sur le moniteur série de la Raspberry Pi.

Résultat attendu doit se rapprocher :





	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	524288	2682144	131072	65536	32768	16384	8192	4096		2048	1024	512	256	128	64	32	16		8	4	2	1
2			MSB[19,12]										LSB[11;4]							XLSB[3;0]		
3																						
4																						
5																						
6																						

Sur le chronogramme, les données relatives à la pression atmosphérique sont les suivantes : 0x48, 0x55, 0x00.

En binaire, cela donne :

- 0x48 → 0b 0100 1000 (MSB)
- 0x55 → 0b 0101 0101 (LSB)
- 0x00 → 0b 0000 (XLSB)

Les autres bits sont insignifiants. Pour obtenir la valeur de la pression, on doit additionner les valeurs correspondantes aux bits activés :

262144 + 32768 + 1024 + 256 + 64 + 16 = 296 272.

Sur le moniteur série, la 14e mesure donne une valeur de 297 152, ce qui montre qu'il y a une bonne corrélation entre les deux. Cela confirme que les données extraites du chronogramme sont bien cohérentes avec les relevés obtenus via le moniteur série de la Raspberry Pi.

Capteur anémomètre:

Register Name	Address	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	Reset state
temp_xlsb	0xFC	temp_xlsb<7:4>				0	0	0	0	0x00
temp_lsb	0xFB	temp_lsb<7:0>								0x00
temp_msb	0xFA	temp_msb<7:0>								0x80
press_xlsb	0xF9	press_xlsb<7:4>				0	0	0	0	0x00
press_lsb	0xF8	press_lsb<7:0>								0x00
press_msb	0xF7	press_msb<7:0>								0x80
config	0xF5	t_sb[2:0]			filter[2:0]				spi3w_en[0]	0x00
ctrl_meas	0xF4	osrs_t[2:0]			osrs_p[2:0]			mode[1:0]		0x00
status	0xF3					measuring[0]			im_update[0]	0x00
reset	0xE0	reset[7:0]								0x00
id	0xD0	chip_id[7:0]								0x58
calib25...calib00	0xA1...0x88	calibration data								individual

Registers:	Reserved registers	Calibration data	Control registers	Data registers	Status registers	Revision	Reset
Type:	do not write	read only	read / write	read only	read only	read only	write only

Conclusion :